UNIVERSITY OF TEHRAN

Electrical and Computer Engineering Department

Computer Aided-Design

CA6

Name:

Mahdi Naieni 810101536

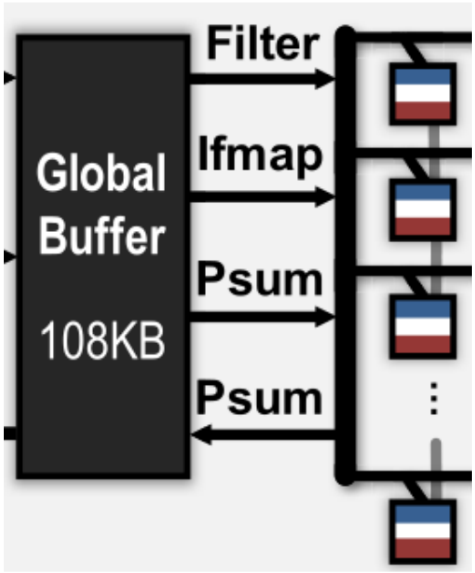
Amin Aghakasiri 810101381

Project Description:

This project is the continuation of CA4 and CA5. In this project, we are supposed to cascade a number of PE's designed in the previous projects.

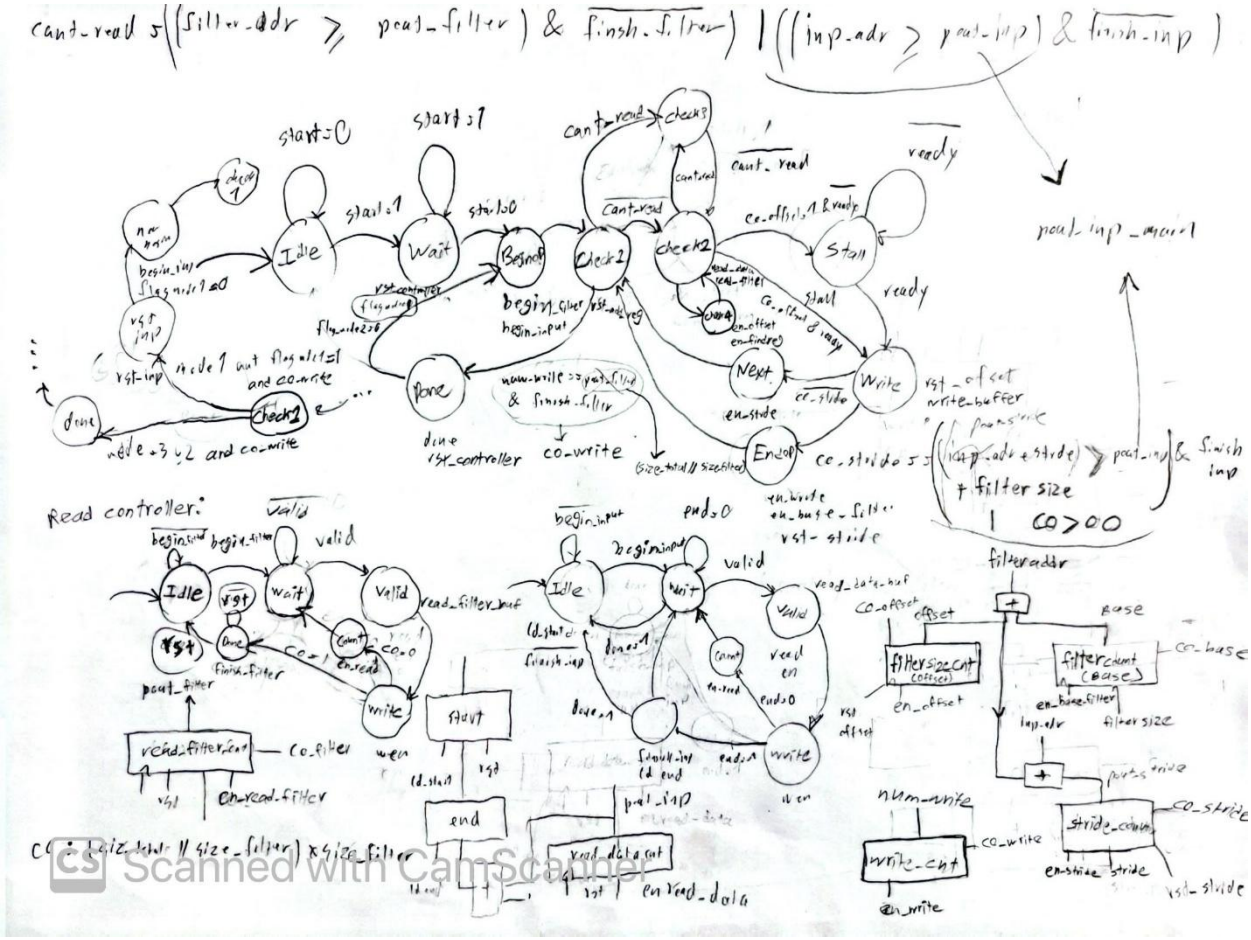
Datapath:

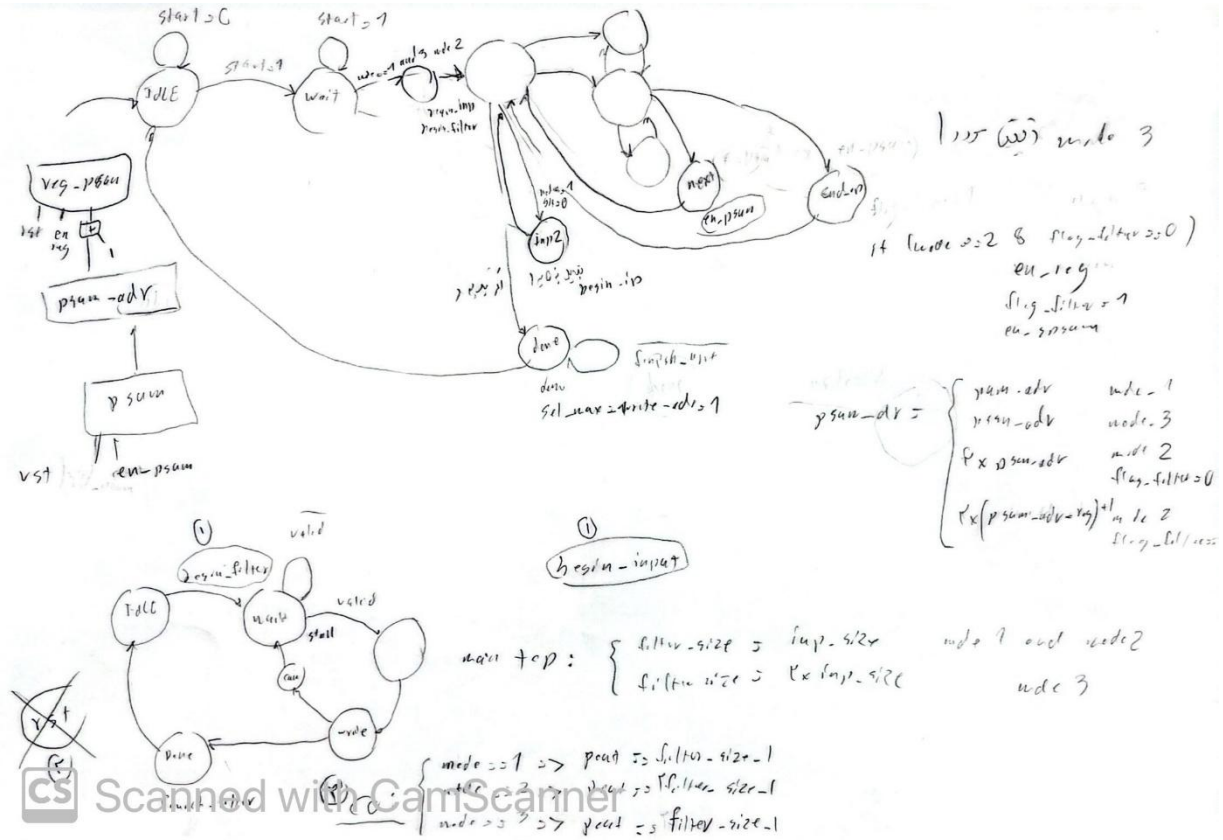


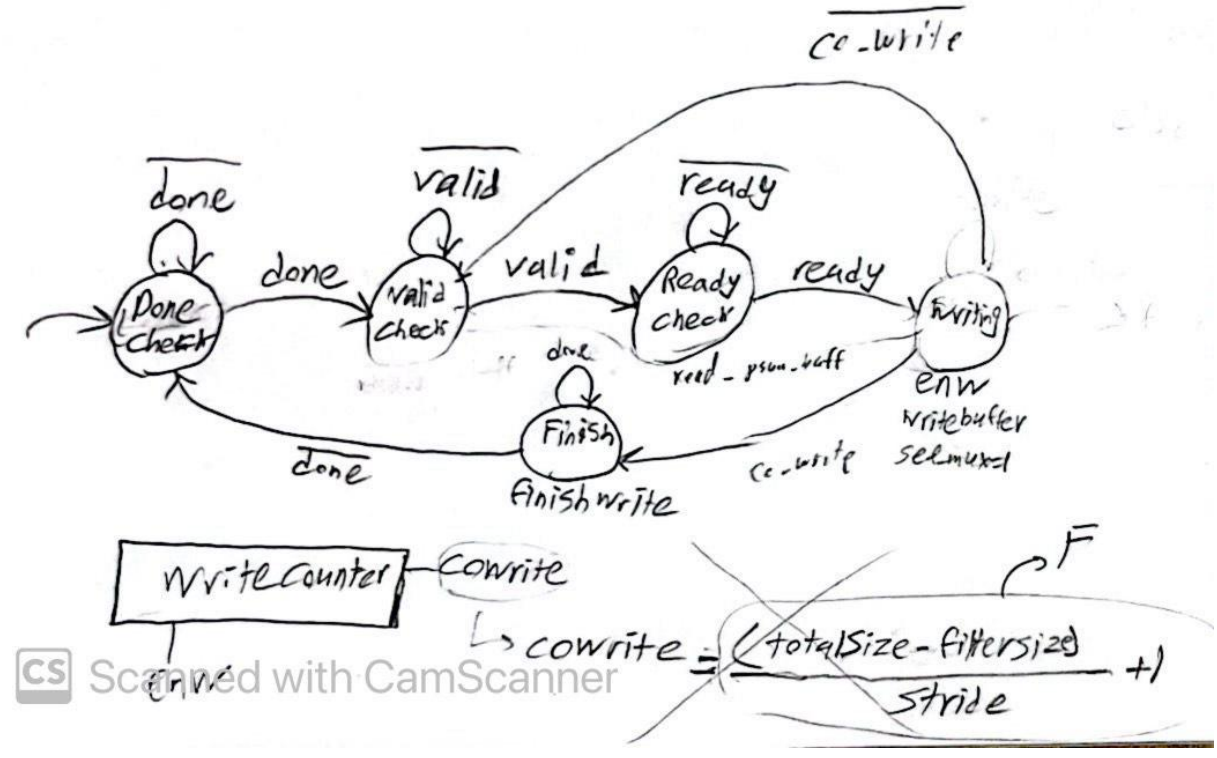


Datapath is the same as the project description.

Controller:


Idle
Wait
Begin
Check1
Check2
Stall
Write
Next
Done
Endop
Read controller:
Valid
Idle
Wait
Valid
Done
Write
Idle
Wait
Valid
Done
Write
end
filterSize cnt
filter cnt
write_cnt
stride_cnt
Scanned with CamScanner





Scanned with CamScanner

Final Top Code:

```
module final_top #(
    parameter N = 3,
    parameter width = 4,
    parameter W_PARAM = 1,
    parameter R_PARAM = 1,
    parameter SIZE_PSUM = 4,
    parameter SIZE_IFMAP = 4,
    parameter BUFFER_SIZE = 16,
    parameter SIZE_FILTER_SRAM = 4,
    parameter ADDR_WIDTH_PSUM = $clog2(SIZE_PSUM),
    parameter ADDR_WIDTH_IFMAP = $clog2(SIZE_IFMAP),
    parameter ADDR_WIDTH_FILTER = $clog2(SIZE_FILTER_SRAM),
    parameter filter_width = $clog2(SIZE_FILTER_SRAM)
) (
    input clk, rst, start, write_en_psum1,
    input [N-1:0] write_en_ifmap, write_en_filter,
    input [1:0] mode,
    input [filter_width-1:0] filter_size,
    input [N*(width+2)-1:0] inp_buf_ifmap,
    input [N*width-1:0]     inp_buf_filter,
    input [width-1:0]       inp_buf_psum1,
    input [ADDR_WIDTH_IFMAP-1:0] stride,
    output done_out
);
  wire [N-1:0] read_en_psum, valid_psum;
  wire [N-1:0] read_en_ifmap, valid_ifmap;
  wire [N-1:0] read_en_filter, valid_filter;
  wire [N-1:0] ready_buf_out, write_en_out, start_signal, end_signal;
  wire [N*width-1:0] inp_psum_temp, inp_ifmap_temp, inp_filter_temp, out_top;
  genvar i;
  generate
    if (N > 0) begin : first_psum
      circular_buffer #(
          .DATA_WIDTH(width),
          .BUFFER_SIZE(BUFFER_SIZE),
          .W_PARAM(W_PARAM),
          .R_PARAM(R_PARAM)
      ) cb_psum_0 (
          .clk(clk),
          .rst(rst),
          .write_en(write_en_psum1),
          .read_en(read_en_psum[0]),
          .inp(inp_buf_psum1),
          .full(),
          .empty(),
          .ready(),
          .valid(valid_psum[0]),
          .data_out(inp_psum_temp[width-1:0])
      );
    end
  endgenerate
```

```
generate
  for (i = 0; i < N; i = i+1) begin : stages
    circular_buffer #(
        .DATA_WIDTH(width+2),
        .BUFFER_SIZE(BUFFER_SIZE),
        .W_PARAM(W_PARAM),
        .R_PARAM(R_PARAM)
    ) cb_ifmap (
        .clk(clk),
        .rst(rst),
        .write_en(write_en_ifmap[i]),
        .read_en(read_en_ifmap[i]),
        .inp( inp_buf_ifmap[ (i+1)*(width+2)-1 : i*(width+2) ] ),
        .full(),
        .empty(),
        .ready(),
        .valid(valid_ifmap[i]),
        .data_out({start_signal[i], end_signal[i],
                   inp_ifmap_temp[(i+1)*width-1 : i*width]})
    );
    circular_buffer #(
        .DATA_WIDTH(width),
        .BUFFER_SIZE(BUFFER_SIZE),
        .W_PARAM(W_PARAM),
        .R_PARAM(R_PARAM)
    ) cb_filter (
        .clk(clk),
        .rst(rst),
        .write_en(write_en_filter[i]),
        .read_en(read_en_filter[i]),
        .inp(inp_buf_filter[(i+1)*width-1 : i*width]),
        .full(),
        .empty(),
        .ready(),
        .valid(valid_filter[i]),
        .data_out(inp_filter_temp[(i+1)*width-1 : i*width])
    );
    if (i == N-1) begin : finall_psum
      top_wrapper #(
          .width(width),
          .SIZE_PSUM(SIZE_PSUM),
          .SIZE_IFMAP(SIZE_IFMAP),
          .SIZE_FILTER_SRAM(SIZE_FILTER_SRAM)
      ) top_inst (
          .clk(clk),
          .rst(rst),
          .start(start),
          .ready(ready_buf_out[i]),
          .valid_ifmap(valid_ifmap[i]),
          .valid_filter(valid_filter[i]),
```

```
            .filter_size(filter_size),
            .inp_buf_ifmap(inp_ifmap_temp[(i+1)*width-1 : i*width]),
            .inp_buf_filter(inp_filter_temp[(i+1)*width-1 : i*width]),
            .inp_buf_psum(inp_psum_temp[(i+1)*width-1 : i*width]),
            .stride(stride),
            .out_buf(out_top[(i+1)*width-1 : i*width]),
            .read_en_filter_buf(read_en_filter[i]),
            .read_en_psum_buf(read_en_psum[i]),
            .read_en_ifmap_buf(read_en_ifmap[i]),
            .write_en_buf(write_en_out[i]),
            .stall(),
            .done_out(done_out)
        );
      end
      else begin : mid_psums
        top_wrapper #(
            .width(width),
            .SIZE_PSUM(SIZE_PSUM),
            .SIZE_IFMAP(SIZE_IFMAP),
            .SIZE_FILTER_SRAM(SIZE_FILTER_SRAM)
        ) top_inst (
            .clk(clk),
            .rst(rst),
            .start(start),
            .ready(ready_buf_out[i]),
            .valid_ifmap(valid_ifmap[i]),
            .valid_filter(valid_filter[i]),
            .valid_psum(valid_psum[i]),
            .end_signal(end_signal[i]),
            .mode(mode),
            .filter_size(filter_size),
            .inp_buf_ifmap(inp_ifmap_temp[(i+1)*width-1 : i*width]),
            .inp_buf_filter(inp_filter_temp[(i+1)*width-1 : i*width]),
            .inp_buf_psum(inp_psum_temp[(i+1)*width-1 : i*width]),
            .stride(stride),
            .out_buf(out_top[(i+1)*width-1 : i*width]),
            .read_en_filter_buf(read_en_filter[i]),
            .read_en_psum_buf(read_en_psum[i]),
            .read_en_ifmap_buf(read_en_ifmap[i]),
            .write_en_buf(write_en_out[i]),
            .stall(),
            .done_out()
        );
      end
    end
  endgenerate
  generate
    for (i = 1; i < N; i = i+1) begin : psums
      circular_buffer #(
          .DATA_WIDTH(width),
          .BUFFER_SIZE(BUFFER_SIZE),
          .W_PARAM(W_PARAM),
          .R_PARAM(R_PARAM)
      ) cb_psum (
          .clk(clk),
          .rst(rst),
          .write_en(write_en_out[i-1]),
          .read_en(read_en_psum[i]),
          .inp(out_top[i*width-1 : (i-1)*width]),
          .full(),
          .empty(),
          .ready(ready_buf_out[i-1]),
          .valid(valid_psum[i]),
          .data_out(inp_psum_temp[(i+1)*width-1 : i*width])
      );
    end
  endgenerate
  circular_buffer #(
      .DATA_WIDTH(width),
      .BUFFER_SIZE(BUFFER_SIZE),
      .W_PARAM(W_PARAM),
      .R_PARAM(R_PARAM)
  ) buffer_out (
      .clk(clk),
      .rst(rst),
      .write_en(write_en_out[N-1]),
      .read_en(),
      .inp(out_top[N*width-1 : (N-1)*width]),
      .full(),
      .empty(),
      .ready(ready_buf_out[N-1]),
      .valid(),
      .data_out()
  );
endmodule
```

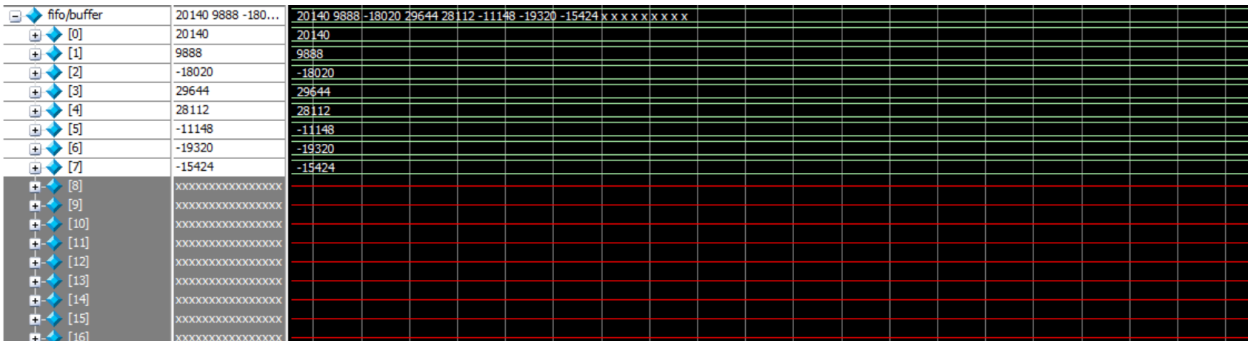
Global Buffer:

```
wire [N*(DATA_WIDTH+2) - 1:0] temp_global_buffer_in;
wire [N*(DATA_WIDTH+2) - 1:0] temp_global_buffer_out;

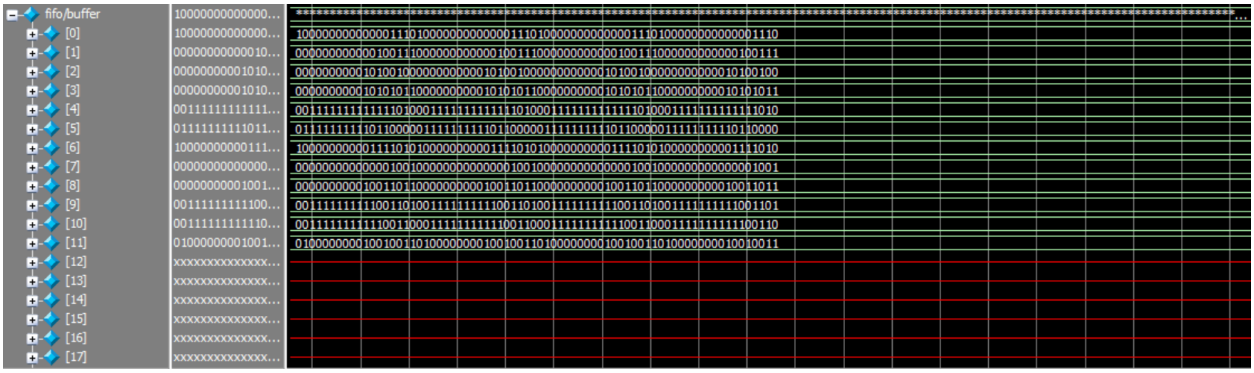
circular_buffer #(.DATA_WIDTH(N * (DATA_WIDTH + 2)), .BUFFER_SIZE(1000), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) global_buffer
                (.clk(clk), .rst(rst), .write_en((write_en_ifmap | write_en_buf)), .read_en(), .inp(temp_global_buffer_in),
                .full(), .empty(), .ready(), .valid(), .data_out(temp_global_buffer_out));

assign temp_global_buffer_in = write_en_ifmap ? inp_buf_ifmap :
                               write_en_filter ? {{(N*2){1'b0}} , inp_buf_filter} :
                               {{(N*(DATA_WIDTH + 2)){1'b0}}};
```

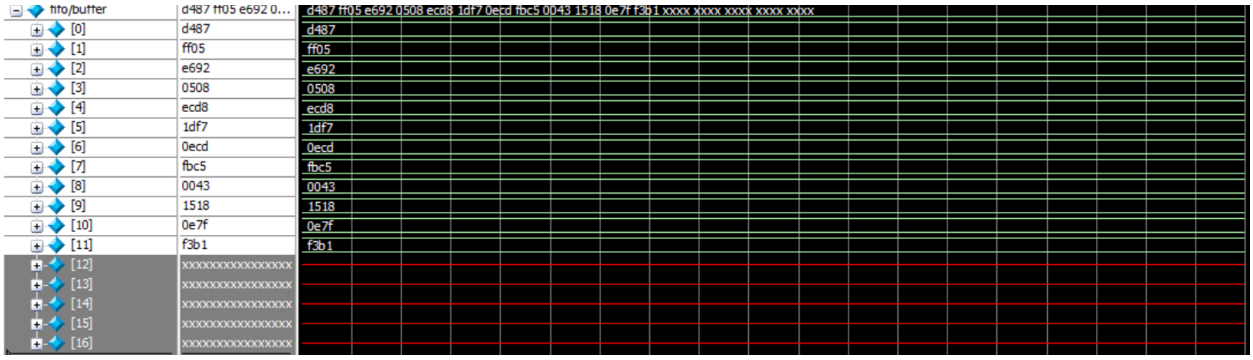
Mode 1 Part 1 Testcase with N = 4:



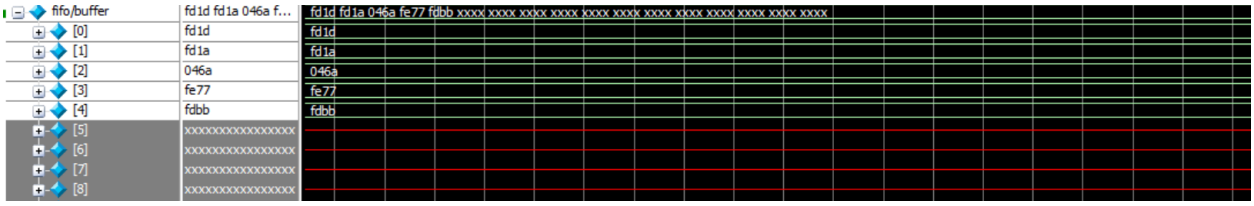
Global Buffer for Above Testcase:



Mode 2:


fifo/buffer
d487 ff05 e692 0...
[0] d487
[1] ff05
[2] e692
[3] 0508
[4] ecd8
[5] 1df7
[6] 0ecd
[7] fbc5
[8] 0043
[9] 1518
[10] 0e7f
[11] f3b1
[12]
[13]
[14]
[15]
[16]
d487 ff05 e692 0508 ecd8 1df7 0ecd fbc5 0043 1518 0e7f f3b1 xxxx xxxx xxxx xxxx xxxx

Mode 3:


fifo/buffer
fd1d fd1a 046a f...
[0] fd1d
[1] fd1a
[2] 046a
[3] fe77
[4] fdbb
[5]
[6]
[7]
[8]
fd1d fd1a 046a fe77 fdbb xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx